

Task API Documentation

REST API for managing tasks — built with Express, TypeScript, and MongoDB (Mongoose).

Base URL: `https://tasks-api-q166.onrender.com`

Note: all endpoint paths below are relative to this base URL — e.g. `GET /tasks` means `GET https://tasks-api-q166.onrender.com/tasks`.

Overview

- All request and response bodies are JSON.
- All endpoints are prefixed at the root — there is no `/api` or version prefix.
- Timestamps (`createdAt` , `updatedAt`) are automatically added to every task.

Authentication

None. There is no authentication or authorization layer — any client can read or write any task. The `user` field is a plain string supplied by the client, not a verified identity.

Data Model: Task

Field	Type	Required	Default	Description
<code>_id</code>	string	auto	—	MongoDB ObjectId, generated on create
<code>title</code>	string	yes	—	Task title/description
<code>completed</code>	boolean	no	<code>false</code>	Whether the task is done
<code>user</code>	string	yes	—	Identifier of the task's owner
<code>createdAt</code>	string	auto	—	ISO timestamp, set on creation
<code>updatedAt</code>	string	auto	—	ISO timestamp, updated on change

Example task object:

```
{
  "_id": "6710a1b2c3d4e5f678901234",
  "title": "Buy groceries",
  "completed": false,
  "user": "alice",
  "createdAt": "2026-09-20T10:15:30.000Z",
  "updatedAt": "2026-09-20T10:15:30.000Z"
}
```

Endpoints

`GET` `/tasks`

Retrieve all tasks, optionally filtered by user.

Query Parameters

Name	Type	Required	Description
------	------	----------	-------------

<code>user</code>	string	no	Return only tasks owned by this user
-------------------	--------	----	--------------------------------------

Response — 200 OK

```
[
  { "_id": "...", "title": "Buy groceries", "completed": false, "user": "alice", "createdAt": "...", "updatedAt": "..." },
  { "_id": "...", "title": "Walk the dog", "completed": true, "user": "alice", "createdAt": "...", "updatedAt": "..." }
]
```

Errors

Status	Cause
500	Database query failed

`GET` `/tasks/:id`

Retrieve a single task by its ID.

Path Parameters

Name	Type	Description
<code>id</code>	string	MongoDB ObjectId of the task

Response — 200 OK

```
{ "_id": "...", "title": "Buy groceries", "completed": false, "user": "alice", "createdAt": "...", "updatedAt": "..." }
```

Errors

Status	Body	Cause
400	<code>{ "error": "Invalid task ID" }</code>	<code>id</code> is not a valid ObjectId
404	<code>{ "error": "Task not found" }</code>	No task exists with that ID
500	<code>{ "error": "Failed to fetch task" }</code>	Database query failed

`POST` `/tasks`

Create a new task.

Request Body

Field	Type	Required	Description
<code>title</code>	string	yes	Task title
<code>user</code>	string	yes	Task owner
<code>completed</code>	boolean	no	Defaults to <code>false</code>

```
{
  "title": "Buy groceries",
  "user": "alice"
}
```

Response — 201 Created

Returns the newly created task, including its generated `_id` and timestamps.

Errors

Status	Body	Cause
400	<code>{ "error": "title and user are required" }</code>	Missing <code>title</code> or <code>user</code>

500	{ "error": "Failed to create task" }	Database write failed
-----	--------------------------------------	-----------------------

PATCH /tasks/:id

Update one or more fields of an existing task.

Path Parameters

Name	Type	Description
id	string	MongoDB ObjectId of the task

Request Body

Any subset of task fields (title , completed , user). Only the fields included are updated.

```
{ "completed": true }
```

Response — 200 OK

Returns the updated task.

Errors

Status	Body	Cause
400	{ "error": "Invalid task ID" }	id is not a valid ObjectId
400	Mongoose validation error	Update fails schema validation
404	{ "error": "Task not found" }	No task exists with that ID
500	{ "error": "Failed to update task" }	Database write failed

DELETE /tasks/:id

Delete a task by its ID.

Path Parameters

Name	Type	Description
id	string	MongoDB ObjectId of the task

Response — 200 OK

```
{ "message": "Task deleted successfully" }
```

Errors

Status	Body	Cause
400	{ "error": "Invalid task ID" }	id is not a valid ObjectId
404	{ "error": "Task not found" }	No task exists with that ID
500	{ "error": "Failed to delete task" }	Database delete failed

Environment Variables

Name	Required	Description
PORT	no	Server port (defaults to 3000)
MONGO_URI	yes	MongoDB connection string; server exits on startup if unset or if the connection fails

Notes for Consumers

- `completed` defaults to `false` if omitted on creation.
- IDs must be valid 24-character MongoDB ObjectId strings; anything else returns `400`.
- There is no pagination on `GET /tasks` — all matching tasks are returned in a single response.
- There is no built-in authentication; if you need to scope access to a real user, add an auth layer in front of this API.